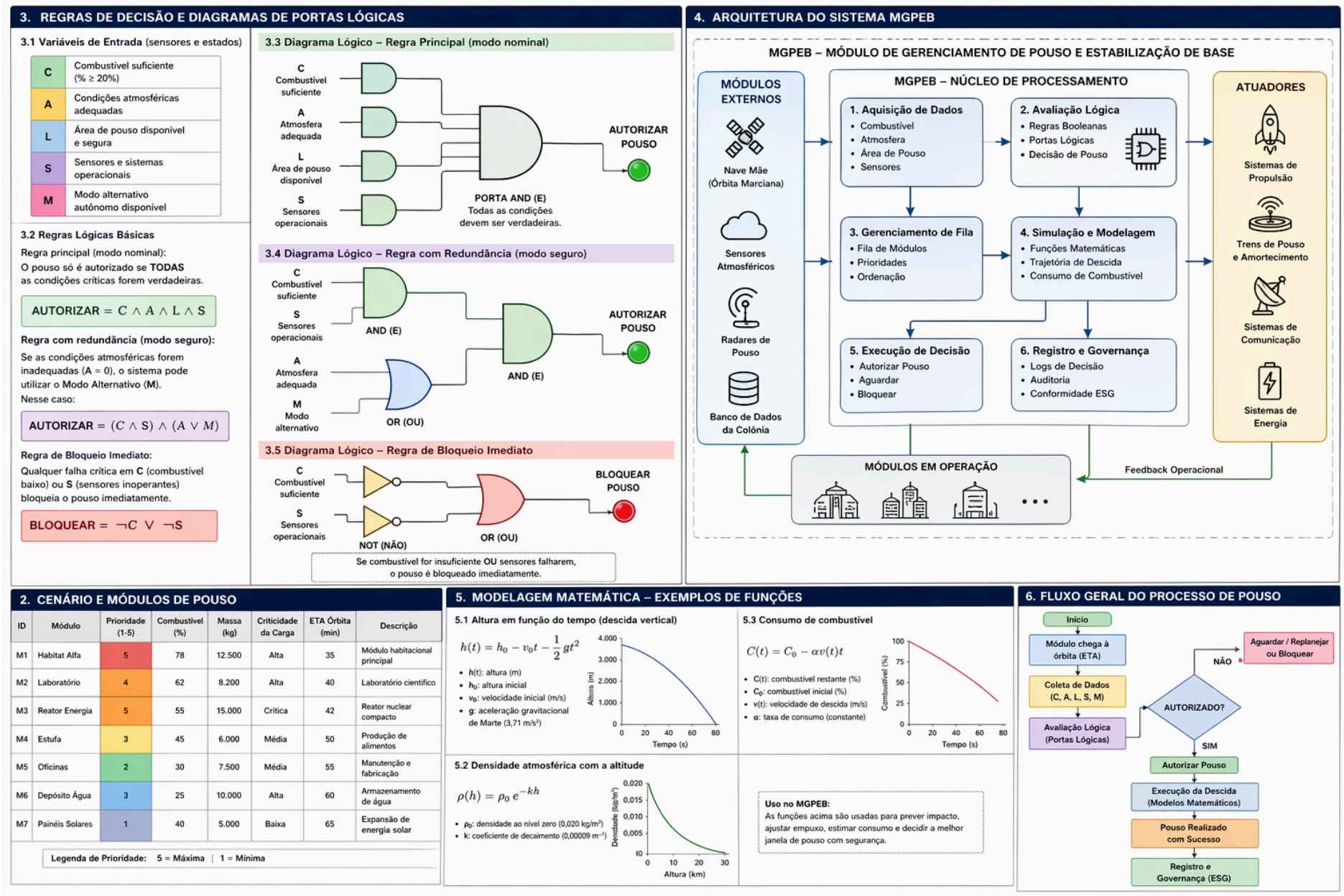
In [ ]:

```
<p>
A missão Aurora Siger propõe a construção de uma colônia autossustentável em Marte, exigindo sistemas altamente confiáveis para operações críticas.
Entre esses sistemas, destaca-se o MGPEB (Módulo de Gerenciamento de Pouso e Estabilização de Base), responsável por coordenar o pouso de módulos, garantir segurança operacional e otimizar o uso de recursos.

CENÁRIO DE POUSO
Contexto Operacional

Durante a fase de expansão da colônia, múltiplos módulos chegam à órbita marciana simultaneamente, exigindo um sistema capaz de:

- priorizar pousos
- evitar colisões
- gerenciar recursos limitados
- responder a condições ambientais

Definição dos Módulos
Cada módulo é caracterizado pelos seguintes atributos:

| Atributo    | Descrição                        |
| ----------- | -------------------------------- |
| Prioridade  | Ordem de importância do pouso    |
| Combustível | Percentual disponível            |
| Massa       | Impacto na estabilidade do pouso |
| Criticidade | Importância da carga             |
| ETA         | Tempo estimado de chegada        |

Organização em Estruturas Lineares
O sistema utiliza:

- Fila (FIFO) → módulos aguardando pouso
- Lista → módulos já processados
- Pilha (LIFO) → eventos críticos

Essa escolha garante:

- previsibilidade
- eficiência
- controle determinístico

REGRAS DE DECISÃO E PORTAS LÓGICAS

Variáveis do Sistema
C → combustível suficiente
A → condições atmosféricas favoráveis
L → área de pouso disponível
S → sensores operacionais

Função Booleana

A autorização de pouso é definida por:

AUTORIZAR= C ∧ A ∧ L ∧ S

Interpretação
AND → todas as condições devem ser verdadeiras
NOT → bloqueio em caso de falha
OR → possibilidade de redundância

Lógica de Segurança (Evolução)

Sistema com redundância:

AUTORIZAR=(C∧S)∧(A∨M)

Onde:

M = modo alternativo/autônomo

Importância

Essa abordagem garante:

decisões determinísticas
auditabilidade
alinhamento com sistemas embarcados

MODELAGEM MATEMÁTICA

A modelagem matemática permite representar fenômenos reais de forma simplificada, mantendo sua relevância operacional e apoiando decisões.

Modelo de Altura

h(t)=h0-vt-1/2gt^2

Aplicações:

previsão de impacto
controle de descida
ativação de retrofoguetes

Velocidade

v(t)=dt/dh

Permite:

monitorar taxa de descida
evitar colisões

Atmosfera Marciana

Modelo exponencial:

ρ(h)=ρ0e^-kh

Aplicações:

ajuste de empuxo
controle aerodinâmico

As funções são integradas à lógica de decisão:

ajuste dinâmico de pouso
detecção de risco
otimização de combustível

ALGORITMOS E COMPLEXIDADE

Busca
busca linear → O(n)
utilizada para identificar módulos críticos

Ordenação
ordenação por prioridade → O(n log n)
otimiza a sequência de pousos

Em sistemas embarcados:

eficiência é essencial
recursos são limitados

CONTEXTUALIZAÇÃO HISTÓRICA E ARQUITETURAL

Evolução da Computação

O desenvolvimento do MGPEB está diretamente ligado à evolução histórica dos computadores:

máquinas mecânicas (Pascal)
cartões perfurados
computadores eletrônicos
sistemas modernos embarcados

Limitações típicas:
```

```
| Limitação | Impacto                   |
| --------- | ------------------------- |
| Memória   | uso de estruturas simples |
| Energia   | algoritmos eficientes     |
| Radiação  | redundância               |

Sistemas Embarcados

O MGPEB segue princípios de:

tempo real
confiabilidade
previsibilidade

ESG E GOVERNANÇA

Ambiental:

seleção consciente de áreas de pouso
minimização de impacto no solo marciano
uso eficiente de energia

Social:

priorização de módulos essenciais à vida
segurança dos tripulantes

Governança:

decisões auditáveis
transparência
logs operacionais

Aplicação Prática:

rastreabilidade de decisões
controle ético de recursos
prevenção de riscos

ESTRUTURAS DE DADOS

Fila:

usada para gerenciar pousos
comportamento FIFO

Pilha:

utilizada para alertas
comportamento LIFO

Lista:

armazenamento geral
flexibilidade

Essas estruturas foram escolhidas por:

simplicidade
eficiência
compatibilidade com sistemas embarcados

# MGPEB - Use Case Diagram

## Atores
- Operador da Missão
- Sistema Autônomo MGPEB
- Sensores Ambientais

## Casos de Uso

### 1. Gerenciar Fila de Pouso
Ator: Sistema MGPEB
Descrição: Organiza módulos por prioridade e criticidade.

### 2. Monitorar Condições Ambientais
Ator: Sensores
Descrição: Fornece dados de atmosfera, área e sensores.

### 3. Simular Descida
Ator: Sistema MGPEB
Descrição: Calcula altura, velocidade e densidade.

### 4. Autorizar Pouso
Ator: Sistema MGPEB
Descrição: Aplica lógica booleana para decisão.

### 5. Gerenciar Alertas
Ator: Sistema MGPEB
Descrição: Identifica falhas críticas (combustível, sensores).

### 6. Gerar Relatório
Ator: Sistema MGPEB
Descrição: Consolida resultados finais.

# Sequence Diagram - MGPEB

Operador -> MGPEB: iniciar_simulacao()
MGPEB -> Fila: ordenar_prioridade()
MGPEB -> Ambiente: gerar_ambiente()

loop para cada módulo
    MGPEB -> Modulo: iniciar_descida()
    Modulo -> Física: calcular_altura()
    Modulo -> Física: calcular_velocidade()
    Modulo -> Ambiente: obter_densidade()

    MGPEB -> Lógica: autorizar_pouso()

    alt pouso autorizado
        MGPEB -> Modulo: status = POUSO
    else
        MGPEB -> Modulo: status = ALERTA
    end
end

MGPEB -> Relatório: gerar()

## Component Diagram - MGPEB

[MGPEB Core]
    |
    |-- [Fila de Prioridade (heapq)]
    |-- [Módulo Física]
    |       |-- altura()
    |       |-- velocidade()
    |       |-- densidade_ar()
    |
    |-- [Módulo Lógica]
    |       |-- autorizar_pouso()
    |
    |-- [Módulo Ambiente]
    |       |-- gerar_ambiente()
    |
    |-- [Módulo Log]
    |
    |-- [Módulo Relatório]

[Entrada]
    -> Sensores
    -> Dados dos módulos
```

```
[Saída]
    -> Status de pouso
    -> Alertas
    -> Logs

    ## Activity Diagram - Fluxo MGPEB

Start
 ↓
Carregar módulos
 ↓
Organizar fila (heap)
 ↓
Loop módulos
 ↓
Simular descida
 ↓
Atualizar altura/velocidade
 ↓
Verificar combustível
 ├── Combustível crítico → ALERTA → Fim módulo
 ↓
Verificar altura < 10
 ├── Não → continuar descida
 ↓
Aplicar lógica booleana
 ├── Verdadeiro → POUSO
 ├── Falso → ALERTA
 ↓
Fim loop
 ↓
Gerar relatório
 ↓
End

## State Diagram - Módulo

[ORBITA]
    ↓
[DESCIDA]
    ↓
[APROXIMACAO]
    ↓
[POUSO]
    ↓
[FINALIZADO]

[DESCIDA] → [ALERTA] (combustível baixo)
[APROXIMACAO] → [ALERTA] (falha lógica)

## Transições de Estado

ORBITA → DESCIDA
    trigger: início da simulação

DESCIDA → APROXIMACAO
    condição: altura < 100

APROXIMACAO → POUSO
    condição: altura < 10 AND lógica OK

APROXIMACAO → ALERTA
    condição: falha lógica

DESCIDA → ALERTA
    condição: combustível < 10

POUSO → FINALIZADO
    condição: pouso concluído

## Métodos do Sistema MGPEB

## Classe Modulo
- registrar(msg)
- __lt__(other)

## Funções Matemáticas
- altura(t, h0, v0)
- velocidade(t, v0)
- densidade_ar(h)

## Lógica
- autorizar_pouso(modulo, ambiente)

## Ambiente
- gerar_ambiente()

## Classe MGPEB
- adicionar_modulo(modulo)
- simular()
- relatorio()


IMPLEMENTAÇÃO EM PYTHON

O protótipo foi desenvolvido utilizando:

classes para modelagem dos módulos
estruturas de dados lineares
funções para lógica de decisão

código modular
fácil manutenção
execução determinística

O MGPEB representa um sistema integrado que combina:

lógica digital
algoritmos
modelagem matemática
engenharia de sistemas

Seu desenvolvimento demonstra como conceitos fundamentais da computação
podem ser aplicados em cenários complexos e críticos, garantindo segurança,
eficiência e sustentabilidade em uma missão espacial.
</p>
```



ID	Módulo	Prioridade (1-5)	Combustível (%)	Massa (kg)	Criticidade da Carga	ETA Órbita (min)	Descrição
M1	Habitat Alfa	5	78	12.500	Alta	35	Módulo habitacional principal
M2	Laboratório	4	62	8.200	Alta	40	Laboratório científico
M3	Reator Energia	5	55	15.000	Crítica	42	Reator nuclear compacto
M4	Estufa	3	45	6.000	Média	50	Produção de alimentos
M5	Oficinas	2	30	7.500	Média	55	Manutenção e fabricação
M6	Depósito Água	3	25	10.000	Alta	60	Armazenamento de água
M7	Painéis Solares	1	40	5.000	Baixa	65	Expansão de energia solar

Legenda de Prioridade: 5 = Máxima | 1 = Mínima

In [22]:

```python
import heapq
import math
import random
from enum import Enum

# ========================
# CORES (terminal)
# ========================
class Cor:
    RESET = "\033[0m"
    RED = "\033[91m"
    GREEN = "\033[92m"
    YELLOW = "\033[93m"
    CYAN = "\033[96m"

# ========================
# ESTADOS
# ========================
class Estado(Enum):
    ORBITA = "orbita"
    DESCIDA = "descida"
    APROXIMACAO = "aproximacao"
    POUSO = "pouso"
    FINALIZADO = "finalizado"
    ALERTA = "alerta"

# ========================
# EVENTOS
# ========================
class Evento(Enum):
    INICIAR_DESCIDA = "iniciar_descida"
    COMBUSTIVEL_CRITICO = "combustivel_critico"
    FALHA_SENSOR = "falha_sensor"
    AREA_INSEGURA = "area_insegura"
    ALTURA_BAIXA = "altura_baixa"
    POUSO_AUTORIZADO = "pouso_autorizado"
    POUSO_ABORTADO = "pouso_abortado"

# ========================
# ALERTAS
# ========================
class TipoAlerta(Enum):
    COMBUSTIVEL_CRITICO = "combustivel_critico"
    POUSO_ABORTADO = "pouso_abortado"
    FALHA_SENSOR = "falha_sensor"
    AREA_NAO_SEGURA = "area_nao_segura"

class Severidade(Enum):
    BAIXA = 1
    MEDIA = 2
    ALTA = 3
    CRITICA = 4

class Alerta:
    def __init__(self, tipo, severidade, descricao, modulo_id, tempo):
        self.tipo = tipo
        self.severidade = severidade
        self.descricao = descricao
        self.modulo_id = modulo_id
        self.tempo = tempo

    def __lt__(self, other):
        return self.severidade.value > other.severidade.value

    def __str__(self):
        cor = Cor.RED if self.severidade == Severidade.CRITICA else Cor.YELLOW
        return f"{cor}[T+{self.tempo:02d}s][{self.severidade.name}] {self.modulo_id} - {self.tipo.value}: {self.descricao}{Cor.RESET}"

# ========================
# FSM
# ========================
class StateMachine:
    def __init__(self, modulo):
        self.modulo = modulo

    def transition(self, novo_estado, motivo, tempo):
        anterior = self.modulo.estado
        self.modulo.estado = novo_estado
        self.modulo.registrar(
            f"TRANSIÇÃO: {anterior.value} → {novo_estado.value} | {motivo}",
            tempo
```

```python
        )

    def processar_evento(self, evento, tempo):
        m = self.modulo

        if m.estado == Estado.ORBITA:
            if evento == Evento.INICIAR_DESCIDA:
                self.transition(Estado.DESCIDA, "Início da descida", tempo)

        elif m.estado == Estado.DESCIDA:
            if evento == Evento.COMBUSTIVEL_CRITICO:
                self.transition(Estado.ALERTA, "Combustível crítico", tempo)

            elif evento == Evento.FALHA_SENSOR:
                self.transition(Estado.ALERTA, "Falha sensor", tempo)

            elif evento == Evento.AREA_INSEGURA:
                self.transition(Estado.ALERTA, "Área insegura", tempo)

            elif evento == Evento.ALTURA_BAIXA:
                self.transition(Estado.APROXIMACAO, "Entrando em aproximação", tempo)

        elif m.estado == Estado.APROXIMACAO:
            if evento == Evento.POUSO_AUTORIZADO:
                self.transition(Estado.POUSO, "Pouso autorizado", tempo)

            elif evento == Evento.POUSO_ABORTADO:
                self.transition(Estado.ALERTA, "Pouso abortado", tempo)

        elif m.estado == Estado.POUSO:
            self.transition(Estado.FINALIZADO, "Pouso concluído", tempo)

# ========================
# MÓDULO
# ========================
class Modulo:
    def __init__(self, id, prioridade, combustivel, massa, criticidade, eta):
        self.id = id
        self.prioridade = prioridade
        self.combustivel = combustivel
        self.massa = massa
        self.criticidade = criticidade
        self.eta = eta

        self.estado = Estado.ORBITA
        self.altura = 1000
        self.velocidade = 0
        self.log = []

        self.fsm = StateMachine(self)

    def registrar(self, msg, tempo):
        entry = f"[T+{tempo:02d}s][{self.id}][{self.estado.name}] {msg}"
        self.log.append(entry)
        print(Cor.CYAN + entry + Cor.RESET)

    def status(self):
        return (
            f"{self.id} | {self.estado.name} | "
            f"ALT={self.altura:.1f}m | VEL={self.velocidade:.1f}m/s | "
            f"FUEL={self.combustivel:.1f}"
        )

    def __lt__(self, other):
        return (self.prioridade, -self.criticidade) < (other.prioridade, -other.criticidade)

# ========================
# FÍSICA
# ========================
def altura(t, h0, v0, g=3.71):
    return max(0, h0 - v0*t - 0.5*g*t**2)

def velocidade(t, v0, g=3.71):
    return v0 + g*t

def densidade_ar(h, rho0=0.02, k=0.0001):
    return rho0 * math.exp(-k*h)

# ========================
# DECISÃO
# ========================
def autorizar_pouso(modulo, ambiente):
    C = modulo.combustivel > 20
    A = ambiente["atmosfera"]
    S = ambiente["sensores"]

    resultado = (C and S) and (A or ambiente["modo_emergencia"])

    print(f"🧠 DECISÃO -> C:{C} S:{S} A:{A} => {resultado}")

    return resultado

# ========================
# AMBIENTE
# ========================
def gerar_ambiente():
    return {
        "atmosfera": random.choice([True, True, False]),
        "area_livre": random.choice([True, True, False]),
        "sensores": random.choice([True, True, True, False]),
        "modo_emergencia": False
    }

# ========================
# SISTEMA
# ========================
class MGPEB:
    def __init__(self):
        self.fila = []
        self.pousados = []
        self.alertas = []

    def adicionar_modulo(self, modulo):
        heapq.heappush(self.fila, modulo)

    def gerar_alerta(self, modulo, tipo, severidade, descricao, tempo):
        alerta = Alerta(tipo, severidade, descricao, modulo.id, tempo)
        print(alerta)
        heapq.heappush(self.alertas, alerta)

    def simular(self):
        tempo = 0

        print("\n🚀 INICIANDO SIMULAÇÃO\n")

        while self.fila:
            ambiente = gerar_ambiente()
            modulo = heapq.heappop(self.fila)

            modulo.fsm.processar_evento(Evento.INICIAR_DESCIDA, tempo)

            for t in range(1, 20):
                modulo.altura = altura(t, modulo.altura, modulo.velocidade)
                modulo.velocidade = velocidade(t, modulo.velocidade)

                rho = densidade_ar(modulo.altura)

                modulo.registrar(
                    f"ALT={modulo.altura:.2f}m | VEL={modulo.velocidade:.2f}m/s | "
                    f"FUEL={modulo.combustivel:.1f} | RHO={rho:.5f}",
                    tempo
                )
```

```python
            modulo.combustivel -= 1.5

            if modulo.combustivel < 10:
                modulo.fsm.processar_evento(Evento.COMBUSTIVEL_CRITICO, tempo)
                self.gerar_alerta(modulo, TipoAlerta.COMBUSTIVEL_CRITICO, Severidade.CRITICA, "Combustível crítico", tempo)
                break

            if not ambiente["sensores"]:
                modulo.fsm.processar_evento(Evento.FALHA_SENSOR, tempo)
                self.gerar_alerta(modulo, TipoAlerta.FALHA_SENSOR, Severidade.ALTA, "Falha de sensor", tempo)
                break

            if not ambiente["area_livre"]:
                modulo.fsm.processar_evento(Evento.AREA_INSEGURA, tempo)
                self.gerar_alerta(modulo, TipoAlerta.AREA_NAO_SEGURA, Severidade.ALTA, "Área insegura", tempo)
                break

            if modulo.altura < 100:
                modulo.fsm.processar_evento(Evento.ALTURA_BAIXA, tempo)

            if modulo.altura < 10:
                if autorizar_pouso(modulo, ambiente):
                    modulo.fsm.processar_evento(Evento.POUSO_AUTORIZADO, tempo)
                    self.pousados.append(modulo)
                else:
                    modulo.fsm.processar_evento(Evento.POUSO_ABORTADO, tempo)
                    self.gerar_alerta(modulo, TipoAlerta.POUSO_ABORTADO, Severidade.ALTA, "Pouso abortado", tempo)
                break

        print("\n📊 STATUS FINAL DO MÓDULO:")
        print(modulo.status())

        tempo += 1

    self.relatorio()

def relatorio(self):
    print("\n" + "="*50)
    print("🚀 RELATÓRIO FINAL")
    print("="*50)

    print(f"\n✅ Pousados: {len(self.pousados)}")
    for m in self.pousados:
        print(f"{Cor.GREEN}✓ {m.id}{Cor.RESET}")

    print(f"\n🚨 Alertas: {len(self.alertas)}")

        print("\n--- ALERTAS PRIORIZADOS ---")
        while self.alertas:
            print(heapq.heappop(self.alertas))

# ========================
# EXECUÇÃO
# ========================
if __name__ == "__main__":
    sistema = MGPEB()

    modulos = [
        Modulo("M1", 1, 40, 1200, 5, 10),
        Modulo("M2", 2, 60, 900, 3, 15),
        Modulo("M3", 1, 25, 1500, 5, 5),
        Modulo("M4", 3, 80, 800, 2, 20)
    ]

    for m in modulos:
        sistema.adicionar_modulo(m)

    sistema.simular()
```

```
🚀 INICIANDO SIMULAÇÃO

[T+00s][M1][DESCIDA] TRANSIÇÃO: orbita → descida | Início da descida
[T+00s][M1][DESCIDA] ALT=998.14m | VEL=3.71m/s | FUEL=40.0 | RHO=0.01810
[T+00s][M1][ALERTA] TRANSIÇÃO: descida → alerta | Falha sensor
[T+00s][ALTA] M1 - falha_sensor: Falha de sensor

📊 STATUS FINAL DO MÓDULO:
M1 | ALERTA | ALT=998.1m | VEL=3.7m/s | FUEL=38.5
[T+01s][M3][DESCIDA] TRANSIÇÃO: orbita → descida | Início da descida
[T+01s][M3][DESCIDA] ALT=998.14m | VEL=3.71m/s | FUEL=25.0 | RHO=0.01810
[T+01s][M3][ALERTA] TRANSIÇÃO: descida → alerta | Área insegura
[T+01s][ALTA] M3 - area_nao_segura: Área insegura

📊 STATUS FINAL DO MÓDULO:
M3 | ALERTA | ALT=998.1m | VEL=3.7m/s | FUEL=23.5
[T+02s][M2][DESCIDA] TRANSIÇÃO: orbita → descida | Início da descida
[T+02s][M2][DESCIDA] ALT=998.14m | VEL=3.71m/s | FUEL=60.0 | RHO=0.01810
[T+02s][M2][ALERTA] TRANSIÇÃO: descida → alerta | Falha sensor
[T+02s][ALTA] M2 - falha_sensor: Falha de sensor

📊 STATUS FINAL DO MÓDULO:
M2 | ALERTA | ALT=998.1m | VEL=3.7m/s | FUEL=58.5
[T+03s][M4][DESCIDA] TRANSIÇÃO: orbita → descida | Início da descida
[T+03s][M4][DESCIDA] ALT=998.14m | VEL=3.71m/s | FUEL=80.0 | RHO=0.01810
[T+03s][M4][ALERTA] TRANSIÇÃO: descida → alerta | Falha sensor
[T+03s][ALTA] M4 - falha_sensor: Falha de sensor

📊 STATUS FINAL DO MÓDULO:
M4 | ALERTA | ALT=998.1m | VEL=3.7m/s | FUEL=78.5

==================================================
🚀 RELATÓRIO FINAL
==================================================

✅ Pousados: 0

🚨 Alertas: 4

--- ALERTAS PRIORIZADOS ---
[T+00s][ALTA] M1 - falha_sensor: Falha de sensor
[T+02s][ALTA] M2 - falha_sensor: Falha de sensor
[T+01s][ALTA] M3 - area_nao_segura: Área insegura
[T+03s][ALTA] M4 - falha_sensor: Falha de sensor
```